

JAVASCRIPT

1. Introduction to JavaScript

- **What is JavaScript?**
 - History and evolution
 - Client-side vs. server-side scripting
- **Setting up your environment**
 - Browsers and Developer Tools
 - Installing a code editor (e.g., VS Code)
 - Basic debugging in Developer Tools

2. JavaScript Basics

- **Syntax and Fundamentals**
 - Variables (`var`, `let`, `const`)
 - Data types (string, number, boolean, null, undefined, symbol, bigint)
 - Operators (arithmetic, assignment, comparison, logical)
- **Control Structures**
 - Conditionals (`if`, `else`, `else if`, `switch`)
 - Loops (`for`, `while`, `do...while`, `for...of`, `for...in`)
- **Functions**
 - Function declarations and expressions
 - Arrow functions
 - Parameters and return values
 - Scope and closures

3. DOM Manipulation

- **Document Object Model (DOM)**
 - Selecting elements (`getElementById`, `querySelector`, etc.)
 - Modifying HTML and CSS (`innerHTML`, `style`, `classList`)
 - Event handling (`addEventListener`, event types)
- **Forms and Input**
 - Form validation
 - Event listeners for forms
 - Dynamic form inputs

4. Intermediate Concepts

- **Arrays and Objects**
 - Creating and manipulating arrays
 - Array methods (`map`, `filter`, `reduce`, `forEach`)
 - Working with objects and object methods
- **Error Handling**
 - `try...catch`
 - `throw` statements
 - Custom error messages
- **Asynchronous JavaScript**
 - Callbacks
 - Promises (`then`, `catch`, `finally`)
 - `async/await`

5. Advanced Topics

- **ES6+ Features**
 - Destructuring
 - Template literals
 - Default parameters
 - Rest and spread operators
 - Modules (`import` and `export`)
- **Object-Oriented Programming (OOP)**
 - Prototypes and inheritance
 - Classes and constructors
 - Encapsulation, inheritance, and polymorphism
- **Functional Programming**
 - Pure functions
 - Higher-order functions
 - Immutability
 - Currying and partial application

6. Working with APIs

- **AJAX and Fetch API**
 - Making HTTP requests
 - Consuming APIs
 - Error handling in API calls
- **JSON**
 - Parsing and stringifying JSON data
 - Working with API responses

7. Browser-Specific Features

- **LocalStorage, SessionStorage, and Cookies**
- **Geolocation API**
- **Web Workers**
- **Canvas and SVG**

8. Testing and Debugging

- **Debugging Techniques**
 - Using breakpoints
 - The `console` object (`log`, `warn`, `error`, `table`)
- **Testing Frameworks**
 - Unit testing with Jest or Mocha
 - Writing test cases

9. JavaScript Build Tools

- **Introduction to npm and Yarn**
- **Module bundlers**
 - Webpack
 - Parcel
- **Transpilers**
 - Babel

10. Frameworks and Libraries

- **Introduction to Popular Frameworks**
 - React.js
 - Vue.js
 - Angular
- **Utility Libraries**
 - Lodash
 - Moment.js (or alternatives like Day.js)

11. Real-World Applications

- **Building Projects**
 - To-Do List
 - Weather App (using an API)
 - Portfolio Website
 - E-commerce Cart
 - Real-time Chat App
- **Version Control**
 - Git basics

- Hosting projects on GitHub

12. Performance Optimization

- **Optimizing JavaScript**
 - Minification and bundling
 - Lazy loading
 - Debouncing and throttling
- **Web Performance APIs**
 - Timing APIs
 - Analyzing performance in DevTools

13. Advanced Topics (Optional)

- **TypeScript**
 - Introduction and usage with JavaScript
- **Node.js**
 - Setting up a backend
 - Creating REST APIs
- **WebAssembly**
 - Introduction and use cases

NODE.JS - Backend combo(javascript+node.js)

Syllabus: Learning Node.js

1. Prerequisites

Before starting with Node.js, ensure you have a good understanding of:

1. **JavaScript Fundamentals**
 - ES6+ features like `let/const`, arrow functions, destructuring, template literals, and promises.
 - Basic programming constructs like loops, arrays, objects, and functions.
2. **Basic Web Development**
 - HTTP and RESTful concepts.

- Basic knowledge of HTML and CSS.

3. Git and Version Control

- Basic Git commands.
- Hosting repositories on GitHub.

2. Introduction to Node.js

- **What is Node.js?**

- Server-side JavaScript runtime.
- Event-driven, non-blocking I/O model.
- Difference between Node.js and traditional server-side technologies.

- **Installing Node.js**

- Setting up Node.js and npm.
- Understanding `node` and `npm` commands.

- **Your First Node.js Application**

- Creating a simple "Hello World" server.

3. Core Concepts

- **Modules and Packages**

- Built-in modules: `fs`, `path`, `http`, `url`, etc.
- Creating custom modules.
- Using npm to manage dependencies.

- **File System (fs)**

- Reading, writing, and deleting files.
- Working with directories.
- Streams and buffers.

- **Events and Event Emitters**

- Understanding the event loop.
- Working with `EventEmitter`.

- **Global Objects**

- `__dirname`, `__filename`, `process`, and more.

4. Building HTTP Servers

- **HTTP Module**

- Creating basic HTTP servers.
- Handling requests and responses.

- Serving static files.
- **Routing**
 - Creating basic routes manually.
 - Understanding query parameters and URL parsing.
- **Middleware**
 - Building custom middleware.
 - Understanding request/response lifecycle.

5. Working with Databases

- **NoSQL Databases**
 - Introduction to MongoDB.
 - Using `mongoose` for schema-based modeling.
- **SQL Databases**
 - Introduction to MySQL/PostgreSQL.
 - Using `sequelize` or `knex.js` for ORM/Query Builders.
- **Database Operations**
 - CRUD operations.
 - Connecting Node.js applications to databases.

6. Asynchronous Programming

- **Callback Functions**
 - Understanding callback patterns and pitfalls.
- **Promises**
 - Chaining promises and handling errors.
- **Async/Await**
 - Writing cleaner asynchronous code.
- **Event Loop**
 - Deep dive into the event loop and its phases.

7. Express.js Framework

- **Setting Up Express**
 - Installing and creating your first Express app.
 - Understanding the directory structure.
- **Routing in Express**
 - Defining GET, POST, PUT, DELETE routes.

- Using route parameters and query strings.
- **Middleware**
 - Built-in middleware (`body-parser`, `express.json`).
 - Third-party middleware.
 - Custom middleware.
- **Template Engines**
 - Rendering views with `EJS`, `Pug`, or `Handlebars`.

8. RESTful API Development

- **Building APIs**
 - REST principles.
 - Designing routes for CRUD operations.
- **Handling Authentication**
 - Using JWT (JSON Web Tokens) for authentication.
 - Role-based access control.
- **Error Handling**
 - Centralized error handling.
 - Logging errors with `winston` or `morgan`.
- **API Documentation**
 - Documenting APIs with Swagger or Postman.

9. Advanced Node.js Concepts

- **Streams**
 - Reading and writing streams.
 - Stream piping.
- **Cluster and Child Processes**
 - Understanding clustering for scaling applications.
 - Forking child processes.
- **WebSockets**
 - Real-time communication using `ws` or `Socket.io`.
- **Worker Threads**
 - Multithreading with `worker_threads`.

10. Security Best Practices

- Protecting routes with authentication.

- Securing sensitive data with environment variables (`dotenv`).
- Validating user input.
- Avoiding SQL injection and XSS attacks.
- Rate limiting and preventing brute force attacks.

11. Testing and Debugging

- **Testing Frameworks**
 - Writing tests with `Mocha` and `Chai`.
 - End-to-end testing with tools like `Supertest`.
- **Debugging**
 - Using `console.log` effectively.
 - Debugging with Node.js Inspector.

12. Deployment

- **Hosting**
 - Deploying Node.js apps to platforms like Heroku, AWS, or Vercel.
- **Environment Setup**
 - Configuring production environments.
 - Handling environment variables.
- **Scaling Applications**
 - Load balancing with `pm2` and `nginx`.